



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

COURSE PROJECT

AGRICULTURAL RESOURCE AND PESTICIDE TRACKER

Subject – Structured and Object-Oriented
Programming (BACSE – 104)

Department – Information Technology
(Winter Semester, 2025-26)

Submitted By:

1. D SUBHAM KUMAR – 25BIT0275
2. L JOSHUA JEFFREY – 25BIT0167
3. RUDRA THAKKAR – 25BIT0122

1. Abstract

The Agricultural Resource and Pesticide Tracker is a C++ console-based application designed to assist farmers and agricultural professionals in efficiently monitoring plant health and managing pesticide usage. The system allows users to record individual plant data including plant ID, name, crop season, and infection level. Based on the infection level, the system automatically classifies the plant's condition and recommends appropriate pesticide actions. The application demonstrates core Object-Oriented Programming concepts including inheritance, polymorphism, exception handling, and file I/O operations, making it both functionally practical and educationally significant as a demonstration of C++ OOP principles.

2. Modules

The system is divided into four primary functional modules:

2.1 Plant Data Management Module

This module handles all core CRUD (Create, Read, Update, Delete) operations for plant records. It allows the user to add new plants by entering the plant ID, name, crop season, and infection level. Input validation is performed using C++ exception handling to reject invalid plant IDs (non-positive integers) and out-of-range infection levels. Each plant record is stored in an array of Plant objects for in-memory access.

2.2 Search and Display Module

This module enables the user to search for a specific plant by its unique ID and display its complete details. It also provides a bulk display feature that iterates over all stored plants and prints their information including infection level and crop season. The search is implemented using a linear scan of the plant array.

2.3 Analysis and Report Generation Module

The ReportGenerator class drives this module. It invokes two key operations for every plant: analyzePlant(), which classifies the plant health status into Healthy, Mild Infection, Moderate Infection, or Critical Condition; and reviewPlant(), which is resolved through a base class pointer demonstrating runtime polymorphism. Detailed recommendations for pesticide action and system response are printed for each plant.

2.4 File Storage Module

The FileStorage class manages persistent data storage using C-style file I/O (FILE*, fopen, fprintf, fscanf). It provides three operations: savetofile() overwrites the file with the current state of all plants; load_from_file() reads previously saved plant records back into memory; and add_to_file() appends a new plant record without rewriting the entire file.

3. Class Details

Class	Data Members	Member Functions
plant_base	int pid char pname[50] char cseason[50]	plant_base() — constructor input_base() — input plant data display_base() — display data reviewPlant() — virtual function
Plant (inherits plant_base)	float inflvl	Plant() — constructor addPlant() — add with validation displayPlant() — full display updatePlant(int) — modify record analyzePlant() — classify health reviewPlant() — override virtual
ReportGenerator	(none)	generate(Plant[], int) — generates full analysis report for all plants
FileStorage (inherits Plant)	(inherited)	savetofile(Plant[], int) — write all to file load_from_file(Plant[], int&) — read from file add_to_file(...) — append single record

4. OOP Concepts Used

OOP Concept	Where Used	Reason / Benefit
Inheritance	Plant inherits plant_base; FileStorage inherits Plant	Promotes code reuse. Common attributes (pid, pname, cseason) defined once in the base class and reused across derived classes without duplication.
Polymorphism (Runtime)	reviewPlant() declared virtual in plant_base; overridden in Plant; called via base class pointer in ReportGenerator	Allows the correct reviewPlant() version to be called at runtime based on actual object type, enabling flexible and extensible reporting without changing the ReportGenerator class.
Encapsulation	All data members and related functions grouped inside classes (plant_base, Plant, FileStorage)	Bundles data and behavior together, reducing external dependencies and making the code modular and maintainable.
Exception Handling	try-catch in addPlant() for invalid pid (int) and invalid inflvl (float)	Prevents invalid data from entering the system. Provides specific error messages for different invalid input types and resets values gracefully.
Constructors	plant_base() and Plant() constructors initialize default values	Ensures all objects start in a well-defined, safe state with no garbage values in memory.
File I/O (Object-level)	FileStorage class encapsulates all file operations using C-style I/O	Separates storage concerns from business logic. Data persists across sessions and can be reloaded, simulating a simple database.

5. Complete Program with Comments

```
#include <iostream>

#include <string.h>

#include <stdio.h>

using namespace std;

int count = 0; // Global counter for number of plants

// =====

// BASE CLASS: plant_base

// Stores common plant attributes: ID, name, crop season

// =====

class plant_base {

public:

    int pid;

    char pname[50];

    char cseason[50];

    plant_base() {

        pid = 0;

        strcpy(pname, "");

        strcpy(cseason, "");

    }

    void input_base() {

        cout << "Enter Plant ID: ";

        cin >> pid;

        cout << "Enter Plant Name: ";

        cin >> pname;
```

```

        cout << "Enter Crop Season: ";
        cin >> cseason;
    }

    void display_base() {
        cout << "Plant ID: " << pid << endl;
        cout << "Plant Name: " << pname << endl;
        cout << "Crop Season: " << cseason << endl;
    }

    // Virtual function enables polymorphism - overridden in Plant
    virtual void reviewPlant(){
        cout << "Base class review function" << endl;
    }
};

// =====
// DERIVED CLASS: Plant (inherits plant_base)
// Adds infection level tracking and management operations
// =====

class Plant : public plant_base {
public:
    float inflvl; // Infection level 0-100

    Plant() : plant_base() {
        inflvl = 0.0;
    }
}

```

```

// Adds a plant with exception handling for invalid inputs
void addPlant() {
    try {
        input_base();
        if (pid <= 0) throw pid;
        cout << "Enter Infection Level (0 to 100): ";
        cin >> inflvl;
        if (inflvl < 0 || inflvl > 100) throw inflvl;
    }
    catch (int e) {
        cout << "Error: Invalid Plant ID (" << e << ")! Must be positive." << endl;
        pid = 0;
    }
    catch (float e) {
        cout << "Error: Infection Level (" << e << ") must be between 0 and 100." <<
endl;
        inflvl = 0;
    }
}

void displayPlant() {
    display_base();
    cout << "Infection Level: " << inflvl << "%" << endl;
    cout << "-----" << endl;
}

void updatePlant(int uid) {

```

```

    cout << "Enter New Plant Name: ";
    cin >> pname;
    cout << "Enter New Crop Season: ";
    cin >> cseason;
    cout << "Enter New Infection Level: ";
    cin >> inflvl;
    while (inflvl < 0 || inflvl > 100) {
        cout << "enter valid infection level" << endl;
        cin >> inflvl;
    }
    cout << "Plant Updated\n";
}

// Classifies plant condition based on infection level
void analyzePlant() {
    cout << "\nPlant: " << pname << endl;
    cout << "Condition: ";
    if(inflvl <= 10)    cout << "Healthy\n";
    else if(inflvl <= 30) cout << "Mild Infection\n";
    else if(inflvl <= 65) cout << "Moderate Infection\n";
    else                cout << "Critical Condition\n";
}

// Overrides virtual function - provides detailed review
void reviewPlant() {
    if(inflvl >= 0 && inflvl <= 10) {
        cout << "Status : Healthy / Safe\n";

```

```

        cout << "Action : No pesticide needed\n";
        cout << "System : Continue standard watering\n";
    }
    else if(inflvl <= 30) {
        cout << "Status : Mild Infection\n";
        cout << "Action : Light preventive spray\n";
        cout << "System : Schedule light spraying\n";
    }
    else if(inflvl <= 65) {
        cout << "Status : Moderate Infection\n";
        cout << "Action : Regular pesticide required\n";
        cout << "System : Initiate pesticide dose\n";
    }
    else if(inflvl <= 100) {
        cout << "Status : Critical\n";
        cout << "Action : Maximum pesticide / isolate plant\n";
        cout << "System : Immediate action required\n";
    }
}

};

// =====
// CLASS: ReportGenerator
// Generates analysis reports using polymorphism
// =====

class ReportGenerator {
public:
    void generate(Plant p[], int count) {

```

```

        for(int i = 0; i < count; i++) {
            p[i].analyzePlant();

            plant_base *ptr = &p[i]; // Base pointer to derived object
            ptr->reviewPlant();    // Calls overridden virtual function
        }
    }
} rg;

// =====
// CLASS: FileStorage (inherits Plant)
// Handles persistent file I/O operations
// =====

class FileStorage : public Plant {
public:

    void savetofile(Plant p[], int count) {
        FILE *fp = fopen("plant_data.txt", "w");

        if (fp == NULL) { cout << "File is empty" << endl; return; }

        for (int i = 0; i < count; i++)
            fprintf(fp, "%d %s %s %f \n", p[i].pid, p[i].pname, p[i].cseason, p[i].inflvl);

        fclose(fp);
    }

    void load_from_file(Plant p[], int &i) {
        FILE *fp = fopen("plant_data.txt", "r");

        if (fp == NULL) { cout << "File is empty" << endl; return; }

        i = 0;

        int id; char name[50], season[50]; float level;

```

```

while (fscanf(fp, "%d %s %s %f", &id, name, season, &level) != EOF) {
    p[i].pid = id;
    strcpy(p[i].pname, name);
    strcpy(p[i].cseason, season);
    p[i].inflvl = level;
    i++;
}
}

```

```

void add_to_file(Plant p[], int count, int new_id,
                char new_name[50], char new_season[50], float infection_lvl) {
    FILE *fp = fopen("plant_data.txt", "a");
    if (fp == NULL) { cout << "File is empty" << endl; return; }
    fprintf(fp, "%d %s %s %f", new_id, new_name, new_season, infection_lvl);
    fclose(fp);
}
};

```

```

// =====
// MAIN FUNCTION: Menu-driven interface
// =====

int main() {
    FileStorage ds;
    Plant p[100];
    int ch = 0;

    while (1) {

```

```

cout << "\n1.Add plant\n2.Update plant\n3.Display details";
cout << "\n4.Search Plant\n5.Detailed Analysis\n6.load data from file\n";
cout << "Enter choice (-1 to stop): ";
cin >> ch;

if (ch == 1) {
    p[count++].addPlant();
    ds.savetofile(p, count);
}
else if (ch == 2) {
    int sid; cout << "enter id to update: "; cin >> sid;
    int i, found = 0;
    for (i = 0; i < count; i++)
        if (p[i].pid == sid) { found = 1; break; }
    if (found) { p[i].updatePlant(sid); ds.savetofile(p, count); }
    else cout << "id not found" << endl;
}
else if (ch == 3) {
    for(int i = 0; i < count; i++) p[i].displayPlant();
}
else if (ch == 4) {
    int sid; cout << "Enter Id to search "; cin >> sid;
    int i;
    for(i = 0; i < count; i++)
        if (p[i].pid == sid) break;
    if (i < count) { cout << "Plant found" << endl; p[i].displayPlant(); }
    else cout << "plant with " << sid << " not found" << endl;
}

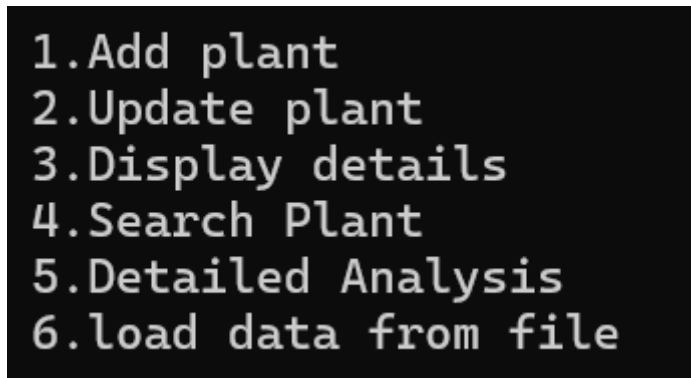
```

```
}  
  
else if (ch == 5) { rg.generate(p, count); }  
  
else if (ch == 6) { ds.load_from_file(p, count); }  
  
else if (ch == -1) break;  
  
}  
  
return 0;  
  
}
```

6. Results and Test Cases

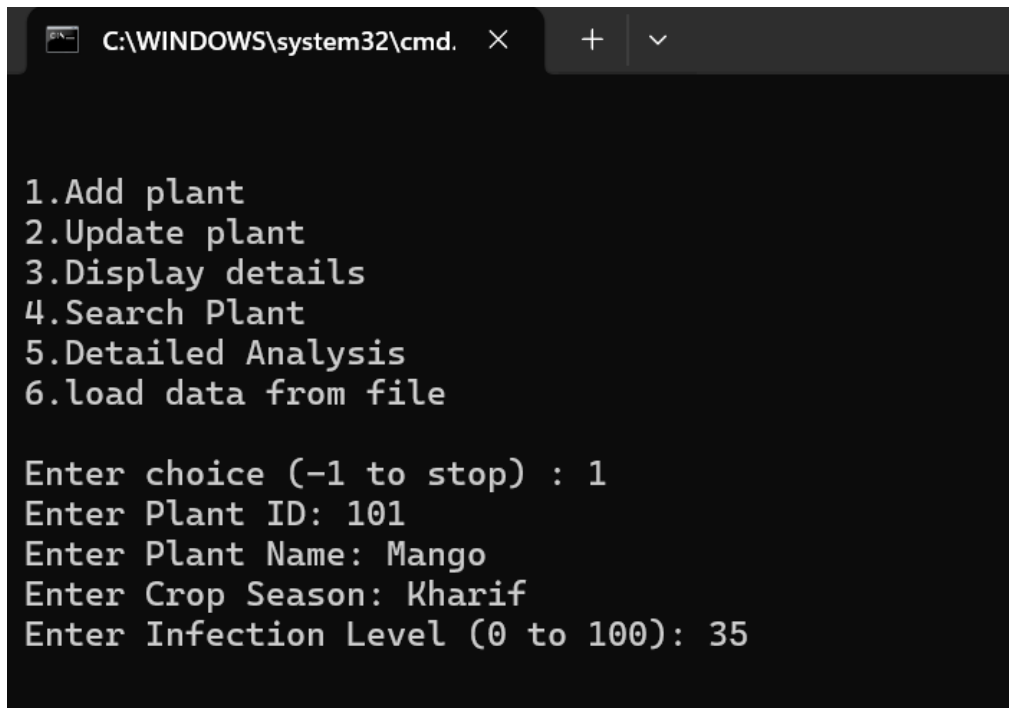
6.1 Menu Display

When the program starts, the following menu is displayed:



```
1.Add plant  
2.Update plant  
3.Display details  
4.Search Plant  
5.Detailed Analysis  
6.load data from file
```

6.2 Test Case 1 — Add a Plant (Valid Input)



```
C:\WINDOWS\system32\cmd.  X  +  v  
  
1.Add plant  
2.Update plant  
3.Display details  
4.Search Plant  
5.Detailed Analysis  
6.load data from file  
  
Enter choice (-1 to stop) : 1  
Enter Plant ID: 101  
Enter Plant Name: Mango  
Enter Crop Season: Kharif  
Enter Infection Level (0 to 100): 35
```

6.3 Test Case 2 — Display Plant

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file

Enter choice (-1 to stop) : 3
Plant ID: 101
Plant Name: Mango
Crop Season: Kharif
Infection Level: 35%
-----
Plant ID: 102
Plant Name: Apple
Crop Season: Rabi
Infection Level: 95%
-----
```

6.4 Test Case 3 – Search Plant

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file

Enter choice (-1 to stop) : 4
Enter Id to search 101
Plant found
Plant ID: 101
Plant Name: Mango
Crop Season: Kharif
Infection Level: 35%
-----
```

6.4 Test Case 4 – Detailed Analysis

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file
```

```
Enter choice (-1 to stop) : 5
```

```
Plant: Mango
Condition: Moderate Infection
Status : Moderate Infection
Action : Regular pesticide required
System : Initiate pesticide dose
```

```
Plant: Apple
Condition: Critical Condition
Status : Critical
Action : Maximum pesticide / isolate plant
System : Immediate action required
```

6.5 Test Case 5 – Update Plant

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file
```

```
Enter choice (-1 to stop) : 2
enter id to update : 102
Enter New Plant Name: Strawberry
Enter New Crop Season: Rabi
Enter New Infection Level: 100
Plant Updated
```

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file
```

```
Enter choice (-1 to stop) : 3
Plant ID: 101
Plant Name: Mango
Crop Season: Kharif
Infection Level: 35%
-----
Plant ID: 102
Plant Name: Strawberry
Crop Season: Rabi
Infection Level: 100%
-----
```

6.6 Test Case 6 – Load Data

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file

Enter choice (-1 to stop) : 6
ID : 101Name : Mangoseason : Khariflevel : 35.000000
ID : 102Name : Strawberryseason : Rabilevel : 100.000000
```

6.7 Test Case 7 – Exit the program

```
1.Add plant
2.Update plant
3.Display details
4.Search Plant
5.Detailed Analysis
6.load data from file

Enter choice (-1 to stop) : -1

Press any key to continue . . .
```